

FORMULARIO DE COMBATE: CC1002 – CONTROL

1

Felipe Colli Olea – FCFM Universidad de Chile – Primavera 2026

1. LA SAGRADA RECETA DE DISEÑO (PUNTAJE FIJO)

Obligatorio en CADA función (hasta 1.5 pts por pregunta):

```
# nombreFuncion: tipo1 tipo2 -> tipoRetorno
# Proposito: Breve descripcion de que calcula
# Ejemplo: nombreFuncion(arg1, arg2) devuelve resultado
def nombreFuncion(param1, param2):
    return resultado
```

```
# Tests (Minimo 1 o 2 por funcion):
assert nombreFuncion(val1, val2) == esperado
assert not funcionBooleana(casoFalso)
assert cerca(funcionReal(val), esperado, 0.0001)
```

Contratos válidos: int, float, num (ambos), str, bool, None (procedimientos).

2. MANIPULACIÓN DE DÍGITOS (SIN USAR STR())

Dado $N \geq 0$ (REGLA: NUNCA usar str(N) si lo prohíben):

- Último dígito (der): $n \% 10$ Quitar der: $n // 10$
- Último par (base 100): $n \% 100$ Quitar par: $n // 100$
- Primeros 2 dígitos (izq): $n // (10 ** (\text{digitos}(n) - 2))$
- Quitar primeros 2 dígitos: $n \% (10 ** (\text{digitos}(n) - 2))$
- Quitar ambos extremos (2 dígitos c/u):
nuevo = $(n \% (10 ** (\text{largo} - 2))) // 100$
- Pegar número a la izquierda: $\text{izq} * (10 ** \text{digitos}(\text{der})) + \text{der}$
- Pegar número a la derecha: $\text{juego} * 100 + \text{nuevaFicha}$

3. PRIMOS Y DIVISORES (CLASE 8)

Verificar si un número es Primo ($O(\sqrt{n})$)

```
# esPrimo: int -> bool
def esPrimo(n):
    if n == 2: return True
    elif n % 2 == 0 or n < 2: return False
    return esPrimoRec(n, 3)
```

```
# esPrimoRec: int int -> bool
def esPrimoRec(n, div=3):
    if div ** 2 > n: return True
    elif n % div == 0: return False
    else: return esPrimoRec(n, div + 2)
```

Generar String con los primeros n Primos

```
# nPrimos: int -> str
def nPrimos(n):
    if n == 0: return ''
    elif n == 1: return '2'
    else: return '2' + nPrimosRec(n - 1, 3)

# nPrimosRec: int int -> str
def nPrimosRec(n, valor):
    if n == 0: return ''
    elif esPrimo(valor):
        return ' ' + str(valor) + nPrimosRec(n - 1, valor + 2)
    else: return nPrimosRec(n, valor + 2)
```

Factores Primos de un Número

```
# menorDivisor: int int -> int
```

```
def menorDivisor(n, div):
    if n % div == 0: return div
    return menorDivisor(n, div + 1)
```

```
# factoresPrimos: int -> str
```

```
def factoresPrimos(n):
    if esPrimo(n): return str(n)
    else:
        div = menorDivisor(n, 2)
        return str(div) + ' ' + str(factoresPrimos(n // div))
```

4. RAÍZ CUADRADA: MÉTODO DE HERÓN (CLASE 7)

```
# cerca: num num num -> bool
```

```
def cerca(x, y, eps=0.0001):
    return abs(x - y) < eps
```

```
# heronRec: num num num -> num
```

```
def heronRec(x, eps, est):
    assert x > 0
    if cerca(est * est, x, eps): return est
    else:
        mejor = (est + (x / est)) / 2.0
        return heronRec(x, eps, mejor)
```

```
# heron: num num -> num
```

```
def heron(x, eps):
    return heronRec(x, eps, x / 2.0)
```

5. TORRES DE HANOI (RECURSIÓN CON STRINGS)

```
# hanoiRec: int str str str -> str
```

```
def hanoiRec(n, partida, medio, final):
    if n == 0: return ''
    t1 = hanoiRec(n - 1, partida, final, medio)
    t2 = str(partida) + ' --> ' + str(final) + '\n'
    t3 = hanoiRec(n - 1, medio, partida, final)
    return t1 + t2 + t3
```

6. GRÁFICOS Y FRACTALES CON TURTLE (CLASE 6)

Primitivas Turtle:

```
import turtle
turtle.forward(dist)    # Avanza
turtle.backward(dist)   # Retrocede
turtle.left(angulo)    # Gira izq en grados
turtle.right(angulo)   # Gira der en grados
turtle.penup()    # Levanta lapiz (no dibuja)
turtle.pendown()    # Baja lapiz (dibuja)
turtle.speed(0)    # Maxima velocidad
turtle.hideturtle()    # Oculta icono tortuga
turtle.done()    # Finaliza dibujo
```

Triángulo de Sierpinski

```
def triangulo(largo):
    turtle.forward(largo); turtle.left(120)
    turtle.forward(largo); turtle.left(120)
    turtle.forward(largo); turtle.left(120)
```

```
def sierpinski(largo):
```

```
    if largo > 4:
        triangulo(largo)
        sierpinski(largo // 2)
        turtle.forward(largo // 2)
        sierpinski(largo // 2)
        turtle.backward(largo // 2); turtle.left(60)
```

```

turtle.forward(largo // 2); turtle.right(60)
sierpinski(largo // 2)
turtle.left(60); turtle.backward(largo // 2); turtle.right(60)

```

Copo de Nieve de Koch

```

def koch(size, min_size):
    if size / 3 < min_size:
        turtle.forward(size)
    else:
        koch(size / 3, min_size); turtle.left(60)
        koch(size / 3, min_size); turtle.right(120)
        koch(size / 3, min_size); turtle.left(60)
        koch(size / 3, min_size)

```

7. PLANTILLAS RECURSIVAS FUNDAMENTALES

Contar Dígitos / Invertir / Mayor Dígito

```

def digitos(n):
    if n < 10: return 1
    return 1 + digitos(n // 10)

def invertir(n):
    if n < 10: return n
    return (n % 10)*(10**digitos(n // 10)) + invertir(n // 10)

def maxDigito(n):
    if n < 10: return n
    r = maxDigito(n // 10); u = n % 10
    return u if u > r else r

```

Procedimiento Recursivo (Mostrar / Imprimir)

```

def mostrar(n):
    if n > 0:
        print("Dato:", n % 100)
        mostrar(n // 100)

```

8. OPERADORES, MÓDULOS Y TRUCOS CLAVE

- **Precedencia Operadores:** `** > *, /, //, % > +, - > Relacionales (<, >, ==, !=) > not > and > or.`

- **División Entera vs Real:** `7 / 2 ->3.5 (float), 7 // 2 ->3 (int), 7 % 2 ->1.`

- **Módulo Math y Random:**

```

import math, random
math.sqrt(x) | math.pi | round(num, 1)
random.randint(a, b) # Entero en [a, b] inclusivo
random.random()     # Real en [0.0, 1.0)

```

- **Redondeo:** `round(3.956, 1) ->4.0, round(2/3, 2) ->0.67.`

- **Input y Conversión limpia:**

```

val = int(input("Ingrese n: "))
txt = input("Nombre? ")
print("Resultado: " + str(val) + " " + txt)

```

- **Retorno Booleano Directo:** Escribe siempre `return (x% 2 == 0)` en vez de `if... return True else return False.`

- **Importar módulos de la pregunta:** Si el problema menciona un módulo ficticio (ej. `domino` o `musical`), escribe: `from domino import *` arriba.